ICT ACADEMY OF KERALA

Summer Internship Report

Full Stack Application Development with ReactJS



Megha K H

Sahrdaya College of Engineering and Technology

25.10.2026

EXECUTIVE SUMMARY

During the full-stack development internship organized by the Information and Communication Technology Academy of Kerala (ICTAK), a comprehensive institutional web application titled **CampusFix** was architected, built, tested, and version-controlled. The project delivers an agile, responsive campus infrastructure maintenance and grievance tracking interface using the MERN stack (MongoDB, Express.js, React, Node.js).

The solution addresses daily campus facility breakdowns and delays through strict role-based data and workflow isolation. Key milestones achieved include building a stateless REST API, enforcing password hashing with bcrypt, issuing signed JSON Web Tokens (JWT) for session management, connecting an AWS-hosted MongoDB Atlas cloud database, and connecting a reactive Vite frontend capable of handling real-time data states.

INTRODUCTION

Full-stack web applications demand seamless integration between client-side user experiences and secure, persistent backend environments. While many generic grievance tools exist, institutional campus maintenance is frequently constrained by manual paper logbooks, lack of progress visibility, and uncoordinated duplicate reporting across departments.

This capstone project addresses this gap by implementing a decoupled, lightweight Single Page Application (SPA). Utilizing an API-first approach, client interactions in React communicate with an Express server over an authenticated REST contract, guaranteeing data privacy, institutional accountability, and sub-second render performance.

OBJECTIVES

- **MERN Architectural Mastery:** Implement separation of concerns across reusable React UI components, Express REST routes, business logic controllers, middleware, and Mongoose document schemas.
- **Security & Role-Based Isolation:** Enforce JWT-based route guarding and bcrypt password hashing (10 salt rounds) to prevent unauthorized status alterations and isolate administrative privilege.
- **State-Driven Interface:** Develop an accessible, mobile-first interface reflecting the four vital UI states: loading skeletons, success views, empty states, and descriptive error banners.
- **Cloud Database & VCS:** Establish a live cloud database on MongoDB Atlas and maintain a structured Git commit history without exposing environment secrets.

SCOPE AND DELIVERABLES

Scope: Role-based account registration (Student/Faculty vs. Facility Admin), automated department mapping, full CRUD ticket lifecycle (Pending → In Progress → Resolved), location-based filtering, and peer upvoting.
Deliverables: Modular React 18 client (`client/`), Express/Node.js REST API (`server/`), MongoDB Atlas schemas, and version-controlled GitHub repository.

METHODOLOGY

The development of the CampusFix Application followed an **Agile-driven MERN architecture**, emphasizing modularity, rapid prototyping, and end-to-end security:

1. **System Design & Architecture:**
   a. Structured as a decoupled 3-tier system: React (Vite) frontend running in the browser, Express/Node.js REST API on port 5000, and a MongoDB Atlas cloud database cluster.

   b. Standardized RESTful routing (GET, POST, PUT, DELETE) with strict HTTP status code handling (200 OK, 201 Created, 400, 401, 403 Forbidden).

2. **Backend & Security Engineering:**
   a. Express.js server connected to MongoDB via Mongoose schemas with relational referencing (`reportedBy` foreign reference in Issue model).

   b. Implemented stateless authentication using JSON Web Tokens (JWT) and `bcryptjs` password hashing (10 salt rounds) through asynchronous Mongoose pre-save hooks.

   c. Route protection enforced via custom middleware validating `Authorization: Bearer <token>` headers and checking user authorization roles.

3. **Frontend Implementation:**
   a. React (Vite) single-page application using controlled components and React hooks (`useState`, `useEffect`, `useContext`).

   b. Complete lifecycle feedback with 4 core UI states: **Loading, Success, Empty, and Error**.

   c. Centralized API service layer injecting JWTs and gracefully handling server error responses.

4. **Testing & Verification:**
   a. API endpoints validated for edge cases (missing fields, duplicate accounts, unauthorized access, CRUD verification).

   b. Cross-browser UI testing to ensure responsiveness, input validation, and secure session persistence.

PROJECT ACTIVITIES

- **Database Modeling:** Formulated normalized schemas in Mongoose. User schema enforces unique email indexes, while Issue models store location, category, urgency, and automated timestamps.
- **Authentication Flow:** Created modular auth middleware extracting Bearer tokens from incoming headers. Passwords are salted using bcrypt before persistence.
- **Frontend Engineering:** Built decoupled React components including Navbar, StatusBadge, PriorityBadge, StateFeedback, Dashboard, NewTicket, Login, and Register.
- **Security & CORS Setup:** Configured explicit CORS headers on the Express server to restrict access while enabling client synchronization without credential leakage.

RESULTS & FINDINGS

Automated and manual verification tests executed against the live local development server validated all functional requirements, security boundaries, and HTTP response statuses:

Endpoint / Flow	Method	Verification Criteria	Result
/api/auth/register	POST	User persistence, unique email check, bcrypt hash, JWT issue	**Passed (201)**
/api/auth/login	POST	Password match, profile & token returned	**Passed (200)**
/api/issues (No Token)	GET	Protected route blocks unauthenticated access	**Passed (401)**
/api/issues	POST	Creates ticket explicitly bound to req.user._id	**Passed (201)**
/api/issues	GET	Fetches campus tickets with search & category query filters	**Passed (200)**
/api/issues/:id/status (Student)	PUT	Non-admin status alteration blocked by RBAC	**Passed (403)**
/api/issues/:id/status (Admin)	PUT	Admin successfully updates status and posts remarks	**Passed (200)**
/api/issues/:id/upvote	PUT	Atomically toggles user ID in upvotes array	**Passed (200)**
Vite Build Process	CLI	Zero warnings, complete production assets in dist/	**Passed**

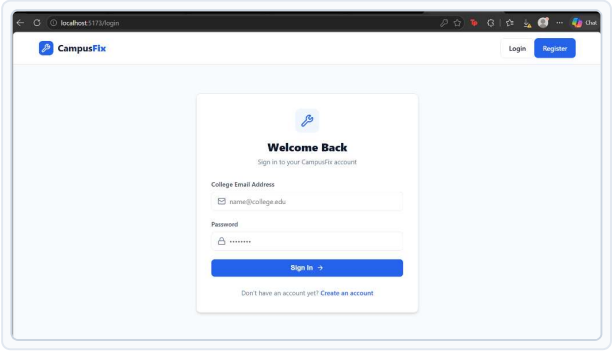


Figure 1: User Login Gateway (/login)
JWT authentication form with client validation and session routing.

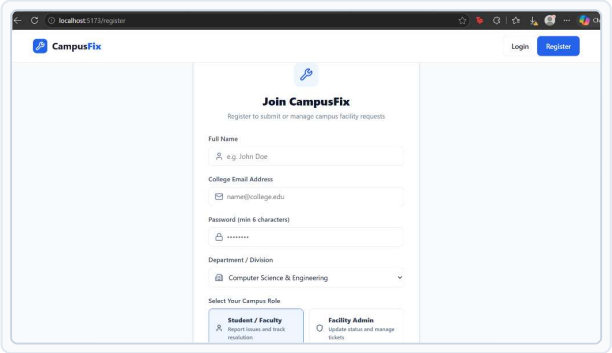


Figure 2: Student Registration (/register)
Account registration capturing department and role declaration.

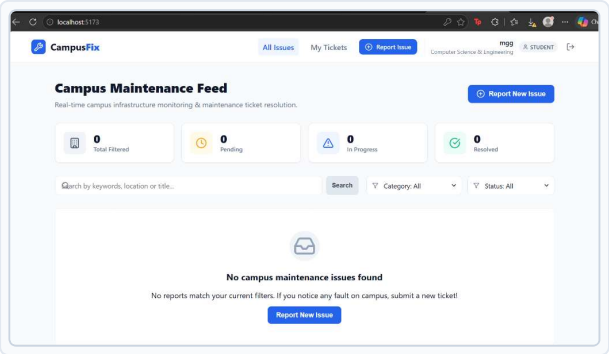


Figure 3: Four UI States — Empty State Feed

Friendly placeholder handling zero-ticket criteria gracefully.

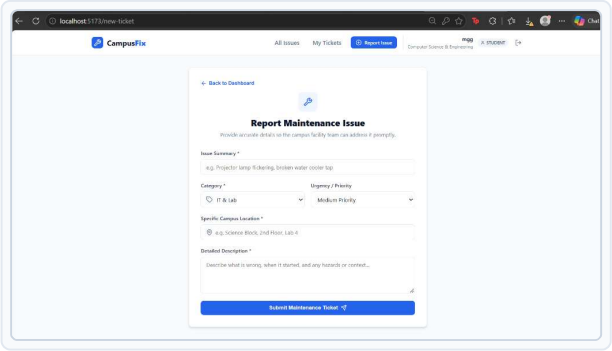


Figure 4: Maintenance Ticket Submission Form

Form capturing category, urgency level, campus location, and description.

RESULTS & SYSTEM SCREENSHOTS (CONT.)

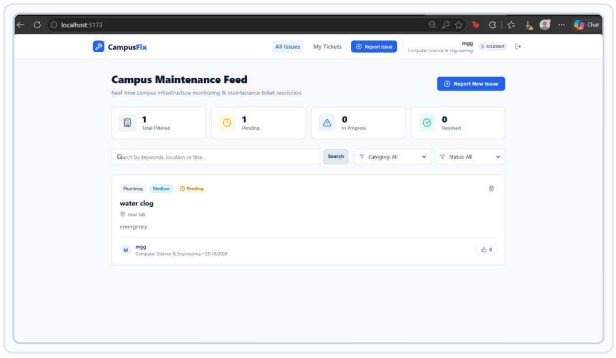


Figure 5: Student View with Active Ticket

Real-time counter (Pending: 1), category badge (Plumbing), and upvote button.

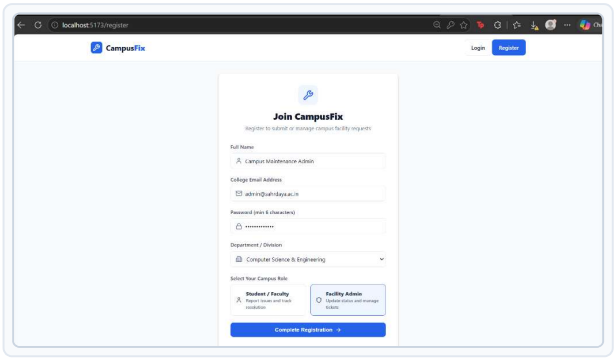


Figure 6: Facility Admin Registration

Provisioning administrative user with Facility Admin role selected.

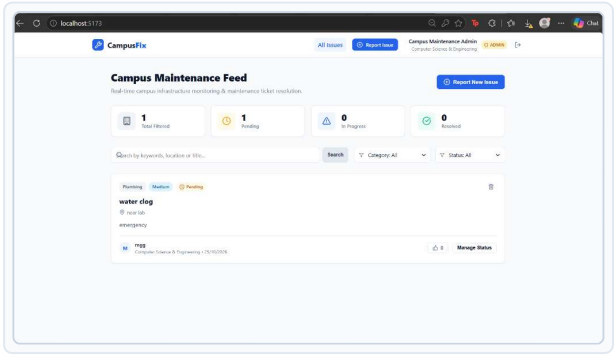


Figure 7: Admin Dashboard & RBAC Control

Gold ADMIN badge in navbar and exclusive "Manage Status" button visible.

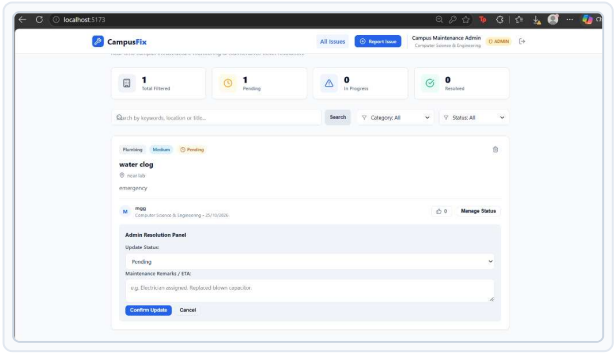


Figure 8: Admin Resolution Drawer Panel

Dropdown to update status and text area for official technician remarks.

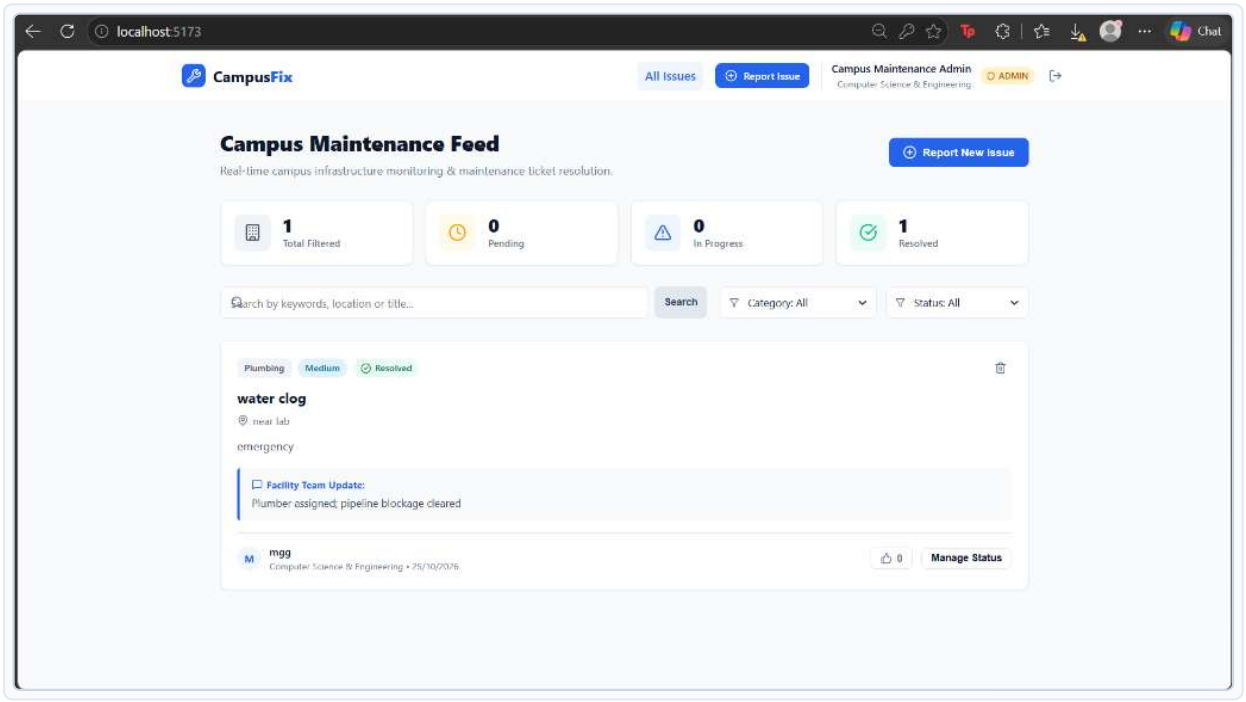


Figure 9: Ticket Resolved with Facility Remarks & Metric Update

Status turns green (Resolved), metrics update (Pending: 0, Resolved: 1), and official resolution notes ("Plumber assigned; pipeline blockage cleared") are appended.

CONCLUSION

The internship culminated in the successful completion of the **CampusFix** Campus Maintenance & Grievance Portal. All core objectives—enforcing REST standards, securing communications via JWT, designing responsive React interfaces with four lifecycle states, establishing role-based access control, and organizing multi-tenant cloud database collections—were fulfilled under the ICTAK curriculum.

This hands-on engagement reinforced core competencies in asynchronous state management, API contract discipline, and production-grade security patterns. By modernizing campus facility tracking into an intuitive, transparent digital workflow, CampusFix demonstrates how full-stack JavaScript architectures deliver tangible operational efficiency in institutional environments.

APPENDIX

Project Architecture:

```
campusfix/
├── client/
│   ├── src/
│   │   ├── components/ (Badges.jsx, Navbar.jsx, ProtectedRoute.jsx, StateFeedback.jsx)
│   │   ├── context/ (AuthContext.jsx)
│   │   ├── pages/ (Dashboard.jsx, Login.jsx, MyTickets.jsx, NewTicket.jsx, Register.jsx)
│   │   ├── services/ (api.js)
│   │   ├── App.css, App.jsx, main.jsx
│   ├── .env.example, package.json
├── server/
│   ├── src/
│   │   ├── config/ (db.js)
│   │   ├── controllers/ (authController.js, issueController.js)
│   │   ├── middleware/ (auth.js, errorMiddleware.js)
│   │   ├── models/ (Issue.js, User.js)
│   │   ├── routes/ (authRoutes.js, issueRoutes.js)
│   │   └── server.js
│   ├── .env.example, package.json
├── .gitignore
└── README.md
```

Project Links:

- **GitHub Repository:** https://github.com/megha225456-ctrl/Campus-Fix
- **Branch:** main
- **Local Development Hosts:** Client: `http://localhost:5173` | Server API: `http://localhost:5000` | Health: `/api/health`